



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
**Proyecto de una Plataforma
Virtual**

Alumno(s):
Gerardo Alexander Díaz León

Introducción

La arquitectura de software es la estructura fundamental sobre la cual se diseña y desarrolla un sistema informático. Elegir una arquitectura adecuada permite mejorar aspectos como el mantenimiento, la escalabilidad, el rendimiento y la organización del código. Existen diversos patrones arquitectónicos que ofrecen soluciones específicas dependiendo de las necesidades del proyecto.

En esta investigación se analizan cinco de las arquitecturas más utilizadas en el desarrollo de software: MVC (Modelo-Vista-Controlador), Arquitectura en Capas, Monolítica, Microservicios y Cliente-Servidor. Se estudian sus características, ventajas, desventajas y casos de uso para comprender sus diferencias y determinar en qué situaciones resulta más conveniente implementar cada una.

CUADRO COMPARATIVO

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar presentación y lógica	Organizar funcionalidades por niveles	Integrar todo en una sola aplicación	Dividir el sistema en servicios independientes	Permitir comunicación entre clientes y servidor
Estructura	Modelo, Vista y Controlador	Capas jerárquicas	Un solo bloque de aplicación	Servicios independientes	Cliente y Servidor
Organización del código	Alta	Alta	Baja en sistemas grandes	Muy alta	Media
Escalabilidad	Media	Media	Baja	Muy alta	Media-Alta
Mantenimiento	Fácil	Fácil	Difícil al crecer	Muy flexible	Moderado
Complejidad	Media	Media	Baja	Alta	Media
Facilidad de implementación	Fácil	Fácil	Muy fácil	Compleja	Moderada

Rendimiento	Alto	Alto	Muy alto	Variable según comunicación	Alto
Reutilización	Alta	Alta	Baja	Muy alta	Media

Casos de uso recomendados	Aplicaciones web	Sistemas empresariales	Proyectos pequeños	Grandes plataformas	Sistemas distribuidos
Ventajas	Separación de responsabilidades	Organización clara	Simplicidad	Escalabilidad y flexibilidad	Centralización de recursos
Desventajas	Curva de aprendizaje	Posible sobrecarga	Difícil mantenimiento	Complejidad y costos altos	Dependencia del servidor

ANÁLISIS

¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

Considero que MVC es el más adecuado porque permite organizar correctamente el código separando la lógica de negocio, la interfaz y el control de las solicitudes. Esto facilita el trabajo colaborativo y el mantenimiento del sistema sin aumentar demasiado la complejidad.

¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Los microservicios permiten escalar cada componente de manera independiente, facilitan el mantenimiento, mejoran la disponibilidad del sistema y permiten realizar despliegues sin afectar toda la aplicación.

¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

Porque ofrece una estructura clara y organizada que facilita el desarrollo, mantenimiento y reutilización del código. Además, es compatible con muchos frameworks modernos como Laravel, ASP.NET MVC y Spring MVC.

¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

Podrían incrementarse los costos de desarrollo, dificultar el mantenimiento, aumentar los tiempos de implementación y generar una complejidad innecesaria para las necesidades reales del proyecto.

¿Crees que existe un patrón mejor que todos los demás? Justifica tu respuesta

No existe un patrón universalmente mejor que los demás. Cada arquitectura está diseñada para resolver problemas específicos y su elección depende del tamaño del proyecto, la cantidad de usuarios, los recursos disponibles y los objetivos del sistema.

CASO PRÁCTICO

Sistema Seleccionado: Ecommerce Arquitectura

Seleccionada: Microservicios Justificación

Número de usuarios:

Un ecommerce puede recibir miles de usuarios simultáneamente, especialmente durante promociones o temporadas de alta demanda.

Escalabilidad:

Los microservicios permiten escalar únicamente los módulos que requieren más recursos, como pagos, catálogo de productos o gestión de pedidos.

Complejidad:

Un ecommerce moderno incluye múltiples funciones como inventario, pagos, envíos y autenticación, por lo que dividir las en servicios independientes facilita su gestión.

Mantenimiento:

Cada servicio puede actualizarse sin afectar al resto del sistema.

Costos:

Aunque la inversión inicial es mayor, a largo plazo permite optimizar recursos y mejorar la disponibilidad del sistema.

Tamaño del equipo de desarrollo:

Diferentes equipos pueden trabajar simultáneamente en distintos servicios sin interferir entre sí.

Conclusión

Durante esta actividad aprendí que la arquitectura de software es uno de los elementos más importantes en el desarrollo de sistemas, ya que determina la forma en que se organiza el código, se comunican los componentes y se gestionan los recursos del proyecto.

La arquitectura que me pareció más interesante fue la de microservicios debido a su capacidad para escalar y adaptarse a sistemas modernos con miles de usuarios. Sin embargo, también comprendí que no siempre es la mejor opción, ya que implica una mayor complejidad y costos de implementación.

La arquitectura influye directamente en la calidad de un sistema porque afecta aspectos como el rendimiento, la mantenibilidad, la seguridad y la escalabilidad. Una buena elección arquitectónica facilita el crecimiento del proyecto y reduce problemas futuros.

Por ello, es fundamental seleccionar adecuadamente la arquitectura desde el inicio del desarrollo, considerando las necesidades reales del sistema, el tamaño del equipo y los recursos disponibles, para garantizar el éxito del proyecto a largo plazo.